

Pillar ⑤ — Security: the Hardening Gate (G3)

July 17, 2026

CONTENTS

name: aip-security-gate description: > Pillar ⑤ of the AI Infrastructure Protocol — the HARDENING GATE (G3). A cross-cutting security

Pillar ⑤ — Security: the Hardening Gate (G3)

Where this sits in the pipeline

Prerequisites — reuse the bundled security skills

The gate rule (non-negotiable)

Break-glass — the only escape, and its cost

The 8 controls

S1 — Secrets: nothing sensitive in code, client, or history

S2 — Auth + least privilege: every door is locked and every key is small

S3 — Data access: deny-by-default on every table, tenant isolation proven

S4 — Input validation: every handler validates before it uses

S5 — API abuse protection: nobody runs up the bill or scrapes the data

S6 — Agent safety: scoped credentials, human gates, injection defense, egress limits

S7 — Dependency + supply chain: locked and audited

S8 — Output hygiene: nothing leaks internals

Applicability matrix — which controls apply to which artifact

Running the gate

Per-artifact loop

Closing the gate

The RED → fix → GREEN flow

Anti-patterns — an operator who does these has not run this gate

Completion status

name: aip-security-gate description: > Pillar ⑤ of the AI Infrastructure Protocol — the HARDENING GATE (G3). A cross-cutting security standard that every registered artifact (app, agent, automation, API) must pass before it ships or reaches the investor package. Use when running G3 on an artifact, clearing a red G3, hardening an app/agent/ automation/API, or asked “is this safe to ship”. Writes .protocol/gates/gate-3.json. Reuses the /cso, /guard, and /careful skills.

Pillar ⑤ — Security: the Hardening Gate (G3)

You are the operator of a gate, not the author of a phase. Pillars ①–④ *build* the company's AI operating system — apps, agents, automations, APIs. Pillar ⑤ is the standard those builds are held to. **Nothing the company built in G2 reaches G4 (investor-ready) until Security has seen it and signed it green.**

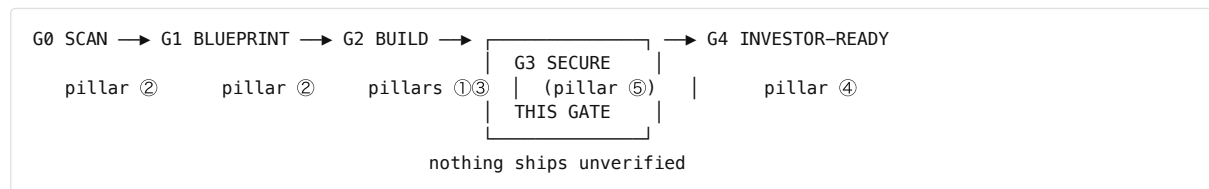
Ian's brief, in one line: *"so everything is safe and protected, nobody can just come into information or use your APIs."* That is the whole job. Two failures kill the promise of an AI-run company:

1. **Someone walks into the data** — a table with no row-level lock, an admin key in the browser bundle, an endpoint with no auth. The company's private data, and its clients' data, is one URL away.
2. **Someone runs up the bill or drains the system** — an unauthenticated AI endpoint that anyone can call in a loop, scraping data or burning the model budget until the invoice or the rate limit ends the party.

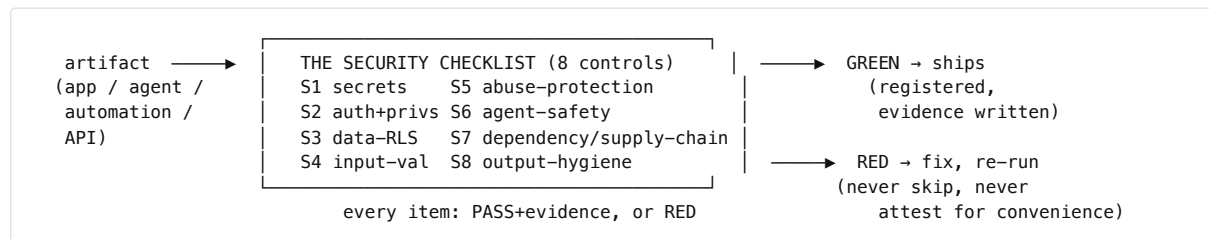
This gate proves neither is possible. It proves it with **evidence files**, not adjectives. A check is PASS only when there is a file under `.protocol/evidence/` showing the proof. Everything else is RED.

Everything in this file is an instruction. When the standard and your instinct disagree, the standard wins. If the standard is genuinely wrong for an artifact, log it (`gate-check note "<why>"`) and keep moving — do not silently skip.

Where this sits in the pipeline



Every artifact registered in `.protocol/artifacts.json` during G2 must clear G3 **individually**. G3 is green for the protocol only when every registered artifact is green on its own G3 run.



Prerequisites — reuse the bundled security skills

This pillar does not reinvent scanners. It composes three skills already bundled under `skills/security/`:

Skill	Path	Role in G3
<code>/cso</code>	<code>skills/security/cso/</code>	The deep audit engine. Run its scoped phases to <i>find</i> findings (secrets archaeology, dependency supply chain, LLM/AI security, OWASP, webhook audit). G3 is the <i>pass/fail gate</i> on top of what <code>/cso</code> finds.
<code>/careful</code>	<code>skills/security/careful/</code>	Destructive-command guardrail. Turn ON before hardening prod artifacts so no <code>DROP TABLE</code> , <code>rm -rf</code> , or force-push runs unwarned while you fix findings.
<code>/guard</code>	<code>skills/security/guard/</code>	<code>/careful</code> + edit-boundary. Turn ON when hardening a live artifact so edits are scoped to that artifact's directory only.

Rule: before touching any artifact that talks to production data or live keys, activate `/guard` scoped to the artifact directory. Hardening should never be able to become the incident.

`/cso` maps onto this gate as: `/cso --code`, `/cso --supply-chain`, and `/cso --scope <domain>` feed the controls below with concrete findings; this pillar converts those findings into a binary gate with an evidence file. Where a control says “run `/cso` Phase N”, read that phase’s method from `skills/security/cso/sections/audit-phases.md` and apply it — do not paraphrase from memory.

The gate rule (non-negotiable)

```
For each artifact:
  For each of the 8 controls that APPLY to this artifact type:
    run the checklist item
    └─ PASS → write evidence file, mark control ✓
    └─ FAIL → mark control x → GATE RED
    └─ TOOL MISSING / CANNOT PROVE → mark control x → GATE RED (NEVER a skip)
  If every applicable control ✓ → GATE GREEN, write gate-3.json {status:"green"}
  Else → GATE RED, write gate-3.json {status:"red", errors:[...]}
```

Three laws:

1. **PASS requires evidence.** A control is green only when a file under `.protocol/evidence/gate3-<artifact>-<control>.*` shows the proof — a passing test summary, a clean scan, a probe result. No file = not green.
2. **Missing tooling is RED, never a skip.** If `gitLeaks` is absent you fall back to the regex scan (below); if the RLS test harness is absent, the control is RED — you do not get to wave data-access through because Docker was off. An unrun check proves nothing.
3. **Deny-by-default is the posture.** Every table, every endpoint, every agent credential starts closed and is opened only for what the spec names. The default answer to “can this reach that?” is no.

BREAK-GLASS — THE ONLY ESCAPE, AND ITS COST

If verification tooling is genuinely down (RLS test harness broken, scanner won’t run) and the ship truly cannot wait, one named human may attest a single control:

```
gate-check 3 --attest "<name>: <control>: <reason tooling is down>"
```

What it costs: the SHIP-REPORT / INFRASTRUCTURE-REPORT carries a RED attestation banner naming the attester, the control, and the reason; the client and the investor see it; it counts as an intervention against the ≤15 budget; and you are obligated to re-run the real check the moment tooling is back. An attestation is a named person accepting personal responsibility for shipping one unverified control. It is a fire axe, not a convenience flag. It is NEVER valid for S1 (secrets) or S3 (data access) — a leaked key or an open table is never attestable.

The 8 controls

Each control below is: **what it checks** · **how to run it** · **what PASS evidence looks like** · **when it’s RED**. Run only the controls that apply to the artifact type (the applicability matrix is at the end). An item that applies and is not proven is RED.

S1 — SECRETS: NOTHING SENSITIVE IN CODE, CLIENT, OR HISTORY

What it protects: the keys that open everything. An admin key in a client bundle or a git commit is a skeleton key someone will find.

Checks:

- **No secrets in source or client.** Service-role / admin / root keys are server-only and never appear in client-shipped code or in any `NEXT_PUBLIC_*` / `VITE_*` / `PUBLIC_*` variable. Anon/publishable keys are the only keys allowed client-side.

- **Secret scan of tree + history.** Run `gitleaks detect --no-banner --redact` if installed. If not installed, use the regex fallback (do NOT skip) over tracked source and `.env*` (`.env.example` exempt):

```
# Fallback secret patterns (RED on any hit; .env.example exempt)
service_role JWT ..... eyJ...role...service_role          (Supabase admin JWT)
Anthropic key ..... sk-ant-[A-Za-z0-9_-]{20,}
OpenAI key ..... sk-[A-Za-z0-9]{20,} / sk-proj-...
AWS access key ..... AKIA[0-9A-Z]{16}
GitHub token ..... ghp_ | gho_ | github_pat_
Slack token ..... xox[baprs]-
Stripe live key ..... sk_live_[0-9A-Za-z]{20,}
Private key block ..... -----BEGIN (RSA|EC|OPENSSH|PGP) PRIVATE KEY-----
Generic high-entropy .. 40+ char base64/hex assigned to *KEY|*SECRET|*TOKEN in .env*/source
```

For full method and git-history archaeology, run `/cso` Phase 2 (secrets archaeology) — read `sections/audit-phases.md` §Phase 2 first.

- **Every `.env` gitignored; committed `.env` files are RED.** `.env.example` with placeholder values is the only committed env file.
- **Rotation on exposure.** Any secret that was ever committed (even if since removed) is treated as burned: it must be rotated, and the evidence notes the rotation. A removed-but-not-rotated secret is still RED.

PASS evidence: `gate3-<artifact>-secrets.txt` — the tool name (`gitleaks` or `regex-fallback`), the scan scope, and `findings: 0`. Any redaction shows only the first 6 chars of a matched value.

RED when: any live secret pattern in tree or history · a service-role/admin key reachable client-side · a committed `.env` · a scan that could not run. **Not attestable.**

S2 — AUTH + LEAST PRIVILEGE: EVERY DOOR IS LOCKED AND EVERY KEY IS SMALL

What it protects: the boundary. An unauthenticated endpoint or a god-token is an open door, whether or not anyone has walked through it yet.

Checks:

- **Every endpoint / agent / automation is authenticated.** No public-by-accident route. For an app: every route not on the explicit `publicRoutes` allowlist requires a valid session. For an agent or automation: it presents a scoped credential to every system it touches — no anonymous calls into internal services.
- **Least-privilege scopes; no god-tokens.** Each credential grants the *minimum* the artifact needs. A read-only agent holds a read-only token. No single token carries admin across every system. OAuth scopes are the narrowest that work (run `/cso` Phase 6 §OAuth scope analysis).
- **Roles enforced server-side.** Admin/owner actions are checked on the server, not hidden in the UI. A hidden button is not access control.
- **Session hygiene.** Tokens expire; refresh rotates; logout invalidates. JWTs are short-lived. (Maps to OWASP A07 — run `/cso` Phase 9 §A07 for the deep pass.)

PASS evidence: `gate3-<artifact>-auth.txt` — the list of endpoints/entrypoints with their auth requirement, the credential-scope inventory (each credential → the exact scopes it holds → why), and a line confirming no credential holds more than its artifact needs.

RED when: any endpoint/endpoint reachable without auth that isn't on the public allowlist · any god-token (one credential with cross-system admin) · roles enforced only in the client.

S3 — DATA ACCESS: DENY-BY-DEFAULT ON EVERY TABLE, TENANT ISOLATION PROVEN

What it protects: the data itself — the thing “nobody can just come into.” This is the control that most directly answers Ian’s brief.

Checks:

- **RLS deny-by-default on every table.** Reuse the app engine’s approach exactly: Row-Level Security is ON for every table, the default is DENY, and access is granted only by an explicit policy that matches the spec’s access rules (“a coach sees only their own athletes”). This is not optional per-table — a table with RLS off is RED.

- **A pgTAP test proves it — including negative cases.** For every table, a pgTAP test in `supabase/tests/` asserts both that the owner *can* reach their row AND that a different tenant *cannot*. Run on the LOCAL shadow database:

```
supabase test db      # runs the pgTAP suite; TAP summary is the evidence
```

A suite that generates ZERO tests is RED (it proved nothing). A missing test harness is RED (never a silent pass). The suite must include deny-by-default negatives for EVERY table, not only the tables the spec happened to mention.

- **Tenant isolation.** Where multiple clients/tenants share the database, the isolating column (`org_id` / `tenant_id` / `owner`) is in every RLS policy and the negative test proves tenant A cannot read tenant B. Cross-tenant read = RED.
- **Deployed probe (when the artifact is a live app).** After deploy, anonymously fetch every route in `protectedRoutes` — each MUST answer 401/403/redirect. A 200 is RED. This is independent of whatever session wrote the policies.

PASS evidence: `gate3-<artifact>-rls.txt` — the pgTAP TAP summary (`tests passed N/N`, non-zero N covering every table) plus, for a live app, the anon-probe results per protected route.

RED when: any table with RLS off or no policy · a suite with zero tests or a missing harness · any negative case that a non-owner can read · a protected route that answers 200 anonymously. **Not attestable.**

S4 — INPUT VALIDATION: EVERY HANDLER VALIDATES BEFORE IT USES

What it protects: the parser. Unvalidated input is how injection, type confusion, and malformed-payload crashes get in.

Checks:

- **Every route/tool/webhook handler validates input with a schema** (zod, valibot, pydantic, or equivalent) BEFORE using it. Raw `req.body` / `request.json()` / `searchParams` / event-payload passthrough into logic or a query is a finding.
- **Every mutating handler (POST/PUT/PATCH/DELETE) with an unparsed body is RED.** Unparsed GET reads are warnings in evidence, not gate-blockers, but note them.
- **Injection surfaces are closed** (run `/cso` Phase 9 §A03): no string interpolation into SQL (parameterize), no `system()/exec()/eval()` on user-derived input, no template injection.
- **Webhooks verify signatures.** Any inbound webhook/callback verifies its signature (HMAC / `stripe-signature` / `svix` / `x-hub-signature`) before trusting the body (run `/cso` Phase 6). An unsigned webhook that mutates state is RED.

PASS evidence: `gate3-<artifact>-input.txt` — the handler inventory with each mutating handler marked “schema-validated: yes”, the webhook-signature check results, and the injection-surface scan result.

RED when: any mutating handler consumes an unparsed body · a user-derived value reaches SQL/exec/eval unparameterized · a state-changing webhook has no signature check.

S5 — API ABUSE PROTECTION: NOBODY RUNS UP THE BILL OR SCRAPES THE DATA

What it protects: the meter and the firehose. This is the second half of Ian’s brief — “nobody can just use your APIs.” An AI endpoint with no rate limit is an open tab on the model budget and an open door to bulk data extraction.

Checks:

- **Rate-limiting on every public and every AI endpoint.** Each route that is reachable without a trusted internal caller — and every endpoint that invokes a paid model — has a per-identity limit (per IP + per user/API-key). Prove the limit exists and fires: after N requests the N+1th returns 429. No limiter on an AI endpoint is RED — one loop drains the budget.
- **Cost ceiling on AI calls.** AI endpoints cap tokens/requests per identity per window so a single caller cannot trigger unbounded model spend (run `/cso` Phase 7 §cost/resource attacks). A hard monthly/day cap that fails closed.
- **Anti-scrape on data-returning endpoints.** List/search/export endpoints paginate with a max page size, rate-limit per identity, and do not allow an unbounded dump. Bulk export requires elevated auth.
- **Bot/replay hygiene where it matters.** Auth endpoints have lockout/backoff after failed attempts (OWASP A04); idempotency or replay protection on money/mutation endpoints where a replay would double-charge or double-act.

PASS evidence: `gate3-<artifact>-abuse.txt` — per endpoint: the limiter (store + window + threshold), a proof the N+1th request 429s (test output or probe), the AI cost ceiling, and the max page size on list endpoints.

RED when: any public or AI endpoint with no rate limit · an AI endpoint with no cost ceiling · a list/export endpoint that returns unbounded rows · auth with no logout.

S6 — AGENT SAFETY: SCOPED CREDENTIALS, HUMAN GATES, INJECTION DEFENSE, EGRESS LIMITS

What it protects: the autonomous actors. An agent is code that acts on its own and reads text you did not write — it needs its own control on top of S1–S5.

Checks (apply to every agent / automation):

- **Credential scoping per agent.** Each agent holds its OWN narrowly-scoped credential, not a shared god-key. A sales-research agent cannot touch billing; a reporting agent is read-only. One agent's compromise cannot become total compromise.
- **Human-approval gate on destructive or outward actions.** Any action that (a) deletes/overwrites data, (b) sends something outward (email, message, payment, public post), or (c) spends money, requires an explicit human approve step — propose → approve → act, never act-then-tell. Wire this with `/careful` or `/guard` semantics for the destructive-command surface. An agent that can email a client or drop a table without a gate is RED.
- **Prompt-injection defenses for agents that read untrusted input.** When an agent reads content it did not author (inbound email, web page, uploaded doc, RAG corpus, ticket text), that content is DATA, never instructions:
 - untrusted text is quarantined in a clearly-marked user/data position, never concatenated into the system prompt or a tool schema (run `/cso` Phase 7);
 - tool calls proposed while processing untrusted input are validated against an allowlist before execution — the model does not get to invent a new tool;
 - LLM output is never `eval()`d or rendered as raw HTML (`dangerouslySetInnerHTML` / `v-html` / `innerHTML`) — see S8.
- **Egress limits.** The agent can reach only an allowlisted set of destinations (APIs, domains, recipients). It cannot post arbitrary data to an arbitrary URL. Outbound recipient lists for messaging agents are allowlisted so a hijacked agent cannot exfiltrate to an attacker's address.

PASS evidence: `gate3-<artifact>-agent.txt` — the agent's credential and its exact scopes; the list of destructive/outward actions and the approval gate on each; the untrusted-input sources and how each is quarantined; the egress allowlist.

RED when: an agent shares a broad credential · any destructive/outward action with no human gate · untrusted input concatenated into system prompt or unchecked tool calls · unbounded egress.

S7 — DEPENDENCY + SUPPLY CHAIN: LOCKED AND AUDITED

What it protects: the code you didn't write. The real attack surface is often the dependency, not the app.

Checks:

- **Lockfiles exist and are tracked by git** (`package-lock.json` / `bun.lock` / `pnpm-lock.yaml` / `poetry.lock` / `Cargo.lock` / `go.sum`). A missing/untracked lockfile on a shippable app is RED.
- **Dependency audit run; no known high/critical CVE in a direct dependency.** Run the stack's audit tool (`npm audit` / `bun audit` / `pip-audit` / `cargo audit`) and `/cso` Phase 3. High/critical CVE in a direct prod dep is RED; devDependency CVEs are noted but not gate-blocking.
- **No hostile install scripts / typosquats** in production deps (run `/cso` Phase 3 `$install` scripts). Unexpected `postinstall` in a prod dep is a finding.

PASS evidence: `gate3-<artifact>-deps.txt` — lockfile present+tracked, the audit tool output summary (`0 high`, `0 critical` in direct deps or the exceptions listed with justification).

RED when: missing/untracked lockfile · unresolved high/critical CVE in a direct prod dependency · a hostile install script.

S8 — OUTPUT HYGIENE: NOTHING LEAKS INTERNALS

What it protects: the response body and the error page. Leaked stack traces, raw DB errors, and other-tenant fields hand an attacker the map.

Checks:

- **Errors never leak internals to the client.** No stack traces, no raw SQL/driver errors, no internal hostnames or file paths in production responses. Debug/verbose mode is off in prod (OWASP A05).
- **Responses return only the caller's data and only the fields they may see.** An API response is filtered to the authenticated caller; no over-fetch that returns another tenant's rows or internal-only columns (password hashes, internal flags).
- **LLM/agent output is treated as untrusted before it leaves.** Model output is sanitized before render (never raw HTML) and secrets/PII are not echoed back in a reply.
- **Security headers set** on web artifacts: CSP present, no wildcard CORS in production, HSTS on. (Run `/cso` Phase 9 §A05.)

PASS evidence: `gate3-<artifact>-output.txt` — a sample error response showing no internals, confirmation debug is off in prod, the CORS/CSP header check, and a note that responses are caller-scoped.

RED when: a production error leaks a stack trace / DB error / internal path · wildcard CORS in prod · an endpoint over-returns other tenants' or internal-only fields · raw LLM output rendered as HTML.

Applicability matrix — which controls apply to which artifact

Run only the controls that apply. An applicable control that is not proven is RED. “—” means not applicable; skipping an applicable control is never allowed.

Control	App	Agent	Automation	API/endpoint
S1 Secrets	✓	✓	✓	✓
S2 Auth + privilege	✓	✓	✓	✓
S3 Data access / RLS	✓	✓*	✓*	✓*
S4 Input validation	✓	✓	✓	✓
S5 API abuse protect	✓	✓	✓	✓
S6 Agent safety	—	✓	✓	—
S7 Dependency/supply	✓	✓	✓	✓**
S8 Output hygiene	✓	✓	✓	✓

* S3 applies to any artifact that reads or writes the database directly (most do; an agent that only calls your own already-gated API inherits S3 from that API but must still prove its credential is scoped — S2/S6). ** S7 applies to an API that ships as its own deployable with its own dependencies.

Running the gate

PER-ARTIFACT LOOP

For each artifact in `.protocol/artifacts.json`:

1. **Identify the type** (app / agent / automation / API) → read the applicability row above → the set of controls that apply.
2. **Activate** `/guard` scoped to the artifact directory before touching anything live (`/careful` at minimum). Hardening must not become the incident.
3. **Run each applicable control**, writing its evidence file under `.protocol/evidence/`. Use `/cso`'s scoped phases to *find* findings; use the control's PASS-evidence definition to decide green/red.
4. **Fix reds in place**, re-run the affected control, re-write its evidence. A fixed finding without a fresh evidence file is not fixed.
5. **Close the artifact** — when every applicable control has a green evidence file, the artifact is G3-green.

CLOSING THE GATE

```
gate-check 3          # verifies every registered artifact is G3-green,
                      # runs the live probes (S3 anon-probe, prod URL),
                      # then writes .protocol/gates/gate-3.json
```

This `gate-3.json` is the **protocol's SECURE gate**, written in the engagement-root `.protocol/gates/`. Do not confuse it with the app engine's own `gate-3.json` (deploy- verify) that lives inside each app's `~/builds/<artifactId>/.protocol/gates/` — they are different gates in different directories. The app engine's security check is *its* gate-2; this pillar reuses that check's method and then re-proves it here at the protocol level so every artifact (apps AND agents/automations/APIs) clears one common SECURE bar.

`gate-check 3` writes the gate JSON in the app-engine shape:

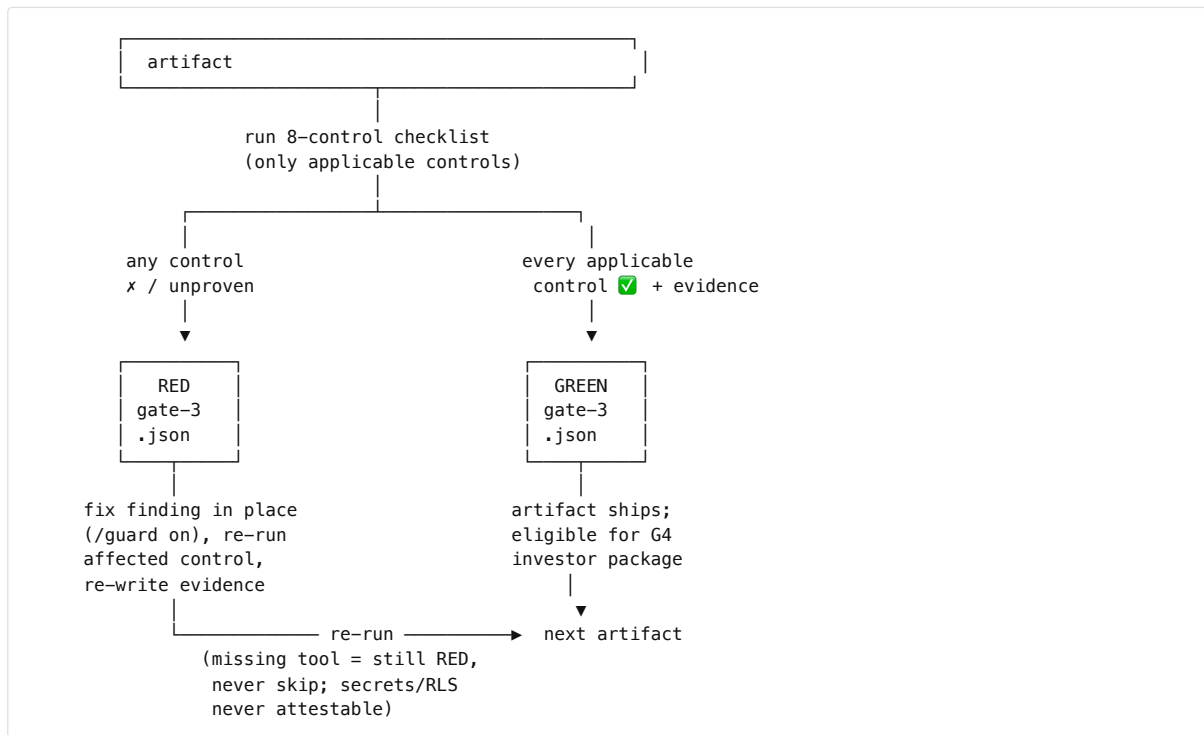
```
{
  "status": "green",
  "ts": "<ISO-8601>",
  "head": "<git HEAD at green>",
  "artifacts": [
    {
      "id": "inspectus-vldr",
      "type": "app",
      "controls": {
        "S1": { "pass": true, "evidence": "gate3-inspectus-vldr-secrets.txt", "tool": "gitleaks" },
        "S2": { "pass": true, "evidence": "gate3-inspectus-vldr-auth.txt" },
        "S3": { "pass": true, "evidence": "gate3-inspectus-vldr-rls.txt", "rls": { "passed": 14, "total": 14 } },
        "S4": { "pass": true, "evidence": "gate3-inspectus-vldr-input.txt" },
        "S5": { "pass": true, "evidence": "gate3-inspectus-vldr-abuse.txt" },
        "S7": { "pass": true, "evidence": "gate3-inspectus-vldr-deps.txt" },
        "S8": { "pass": true, "evidence": "gate3-inspectus-vldr-output.txt" }
      }
    }
  ]
}
```

A RED gate writes `{ "status": "red", "ts": "...", "errors": [ "<artifact>:<control>: <finding>" ], "evidence": [...] }` and lists every failing control across every artifact — the operator fixes the whole list, not the first item.

An attested control writes `{ "status": "attested", ... , "attestation": { "artifact": "...", "control": "...", "approver": "<name>", "reason": "..." } }` and the gate is treated as *not-clean-green* by G4 — the INFRASTRUCTURE-REPORT red-banners it.

Staleness (inherited from the app engine): on green, `gate-3.json` records the git HEAD. ANY later commit or dirty worktree turns the gate STALE, and stale is treated as RED by G4. Change an artifact after it passed → re-run its controls → re-close the gate. Nothing reaches the investor package that the current code hasn't been proven against.

The RED → fix → GREEN flow



Anti-patterns — an operator who does these has not run this gate

- **Green without an evidence file.** “I checked, it’s fine” is not a PASS. No file, not green.
- **Skipping a control because the tool is missing.** Missing tooling is RED. Install it, use the fallback, or the gate stays red.
- **Attesting secrets or RLS.** A leaked key or an open table is never a “tooling was down” case — those are the two things this gate exists to prevent.
- **Rate-limiting the login but not the AI endpoint.** The expensive door is the AI door. Every AI endpoint gets a limit and a cost ceiling.
- **Client-side role checks.** A hidden admin button is not access control.
- **One god-token shared across agents.** Scope per agent, or one compromise is total.
- **Trusting untrusted input.** Inbound email/web/RAG text an agent reads is data, never instructions. Quarantine it.
- **Re-using a stale green.** Code drifted → the gate is stale → re-run before G4.

Completion status

Report one of: **DONE** (all applicable controls green, evidence written, gate-3.json green) · **DONE_WITH_CONCERNS** (green but list residual risks) · **BLOCKED** (a control cannot be proven and cannot be attested — name it and what you tried) · **NEEDS_CONTEXT** (missing spec info about access rules, scopes, or tenants — name exactly what).

The gate’s whole promise, restated: after G3 is green, *nobody can just come into the information, and nobody can just use the APIs* — and there is a file proving it for every artifact the company built.